DEPARTMENT OF

COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

Experiment:5

Student Name: Jetavya Singh Naruka **UID:** 23BCS11171
Branch: BE-CSE **Section/Group:** KRG-3B
Semester: 6 **Date of Performance:**24/02/2026
Subject Name: System Design **Subject Code:** 23CSH-314

- **Aim:** To simulate and understand the complete flow of a URL request from browser to website, including DNS lookup, HTTP/HTTPS request handling, and reverse proxy behavior, using UML modeling and programming simulation.

- **Objective:**
  - To understand the step-by-step process of what happens when you type a URL.
  - To simulate DNS resolution and IP address lookup
  - To model HTTP vs HTTPS request handling
  - To implement reverse proxy behavior
  - To create UML sequence diagrams for request-response flow
  - To build a basic networking simulation in programming language

- **Procedure:-**

```
import time

from urllib.parse import urlparse

# URL MODEL

class URLModel:
```

```
    def __init__(self, url):
        parsed = urlparse(url)
        self.protocol = parsed.scheme if parsed.scheme else "http"
        self.hostname = parsed.hostname
        self.path = parsed.path if parsed.path else "/"
        self.query = parsed.query
        self.port = parsed.port
        if not self.port:
            self.port = 443 if self.protocol == "https" else 80
    def display(self):
        print("\n[Browser] Parsed URL:")
        print(" Protocol :", self.protocol.upper())
        print(" Host     :", self.hostname)
        print(" Path     :", self.path)
        print(" Port     :", self.port)
        if self.query:
            print(" Query    :", self.query)
# DNS RESOLVER
class DNSResolver:
    def __init__(self):
        self.cache = {}
        self.records = {
            "www.example.com": "93.184.216.34",
            "www.google.com": "142.250.183.206",
```

```
        "localhost": "127.0.0.1"
    }
def resolve(self, hostname):
    print("\n[DNS] Resolving:", hostname)
    if hostname in self.cache:
        print("[DNS] Cache hit!")
        return self.cache[hostname]
    print("[DNS] Cache miss. Performing lookup...")
    print("[DNS] Checking local DNS server...")
    time.sleep(0.5)
    print("[DNS] Contacting root server...")
    time.sleep(0.5)
    print("[DNS] Contacting TLD server (.com)...")
    time.sleep(0.5)
    print("[DNS] Contacting authoritative server...")
    time.sleep(0.5)
    ip = self.records.get(hostname)
    if ip:
        self.cache[hostname] = ip
        print("[DNS] Resolved to:", ip)
        return ip
    else:
        print("[DNS] ERROR: Domain not found")
        return None
```

```
# TCP + TLS SIMULATOR
class TCPSimulator:
    def connect(self, ip, port, secure=False):
        print("\n[TCP] Establishing connection to", ip, "on port", port)
        print("[TCP] SYN")
        time.sleep(0.3)
        print("[TCP] SYN-ACK")
        time.sleep(0.3)
        print("[TCP] ACK")
        time.sleep(0.3)
        print("[TCP] Connection Established")
        if secure:
            print("\n[TLS] Starting TLS Handshake")
            print("[TLS] Client Hello")
            time.sleep(0.3)
            print("[TLS] Server Hello + Certificate")
            time.sleep(0.3)
            print("[TLS] Key Exchange")
            time.sleep(0.3)
            print("[TLS] Secure Channel Established")
    def close(self):
        print("\n[TCP] FIN")
        print("[TCP] FIN-ACK")
        print("[TCP] ACK")
```

```
        print("[TCP] Connection Closed")

# REVERSE PROXY

class ReverseProxy:

    def forward(self, path):

        print("\n[Reverse Proxy] Checking routing rules...")

        if "/api" in path:

            backend = "192.168.1.103:3000"

        elif "/app1" in path:

            backend = "192.168.1.101:8080"

        else:

            backend = "192.168.1.100:80"

        print("[Reverse Proxy] Forwarding request to backend:",
backend)

        return backend

# WEB SERVER

class WebServer:

    def handle(self, path):

        print("\n[Web Server] Processing request:", path)

        routes = {

            "/": "<html><h1>Home Page</h1></html>",

            "/about": "<html><h1>About Page</h1></html>",

            "/contact": "<html><h1>Contact Page</h1></html>",

            "/api/data": '{"status":"success","data":[1,2,3]}'

        }
```

```
        if path in routes:
            print("[Web Server] 200 OK")
            return routes[path], 200
        else:
            print("[Web Server] 404 Not Found")
            return "<html><h1>404 Not Found</h1></html>", 404
# BROWSER SIMULATOR
class BrowserSimulator:
    def render(self, content, status):
        print("\n[Browser] Receiving Response")
        print("[Browser] Status Code:", status)
        print("[Browser] Rendering page...")
        time.sleep(0.5)
        print("\n===== PAGE CONTENT =====")
        print(content[:150])
        print("========================")


# MAIN SIMULATION
def simulate():
    print("=== URL-to-Website Request Flow Simulation (Python)
===")
    url_input = input("\nEnter URL: ")
    # Step 1: Parse URL
    url = URLModel(url_input)
```

```
    url.display()
    # Step 2: DNS
    dns = DNSResolver()
    ip = dns.resolve(url.hostname)
    if not ip:
        return
    # Step 3: TCP/TLS
    tcp = TCPSimulator()
    tcp.connect(ip, url.port, secure=(url.protocol == "https"))
    # Step 4: Reverse Proxy (skip if localhost)
    if url.hostname != "localhost":
        proxy = ReverseProxy()
        proxy.forward(url.path)
    # Step 5: Web Server
    server = WebServer()
    content, status = server.handle(url.path)
    # Step 6: Browser Render
    browser = BrowserSimulator()
    browser.render(content, status)
    # Step 7: Close Connection
    tcp.close()
    print("\n=== Simulation Complete ===")
# Run Program
if __name__ == "__main__":
```

simulate()

- **Output:**

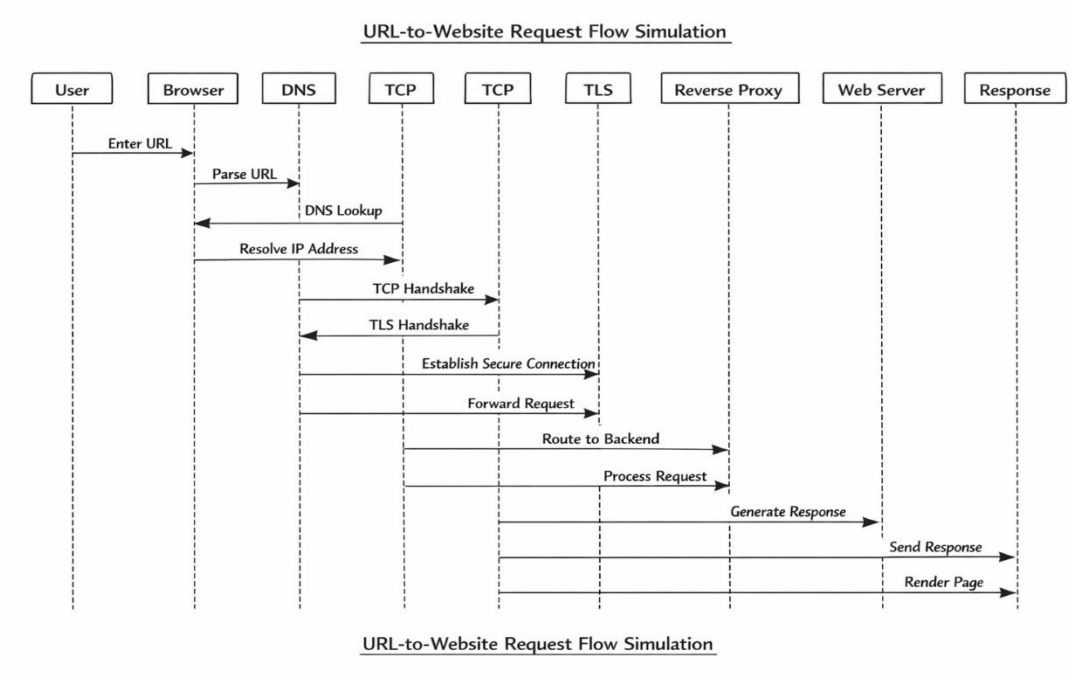

FIG. UML DIAGRAM

```
===== PAGE CONTENT =====
<html><h1>Home Page</h1></html>
========================

[TCP] FIN
[TCP] FIN-ACK
[TCP] ACK
[TCP] Connection Closed

=== Simulation Complete ===
```

```
=== URL-to-Website Request Flow Simulation (Python) ===

Enter URL: http://www.google.com

[Browser] Parsed URL:
 Protocol : HTTP
 Host     : www.google.com
 Path     : /
 Port     : 80

[DNS] Resolving: www.google.com
[DNS] Cache miss. Performing lookup...
[DNS] Checking local DNS server...
[DNS] Contacting root server...
[DNS] Contacting TLD server (.com)...
[DNS] Contacting authoritative server...
[DNS] Resolved to: 142.250.183.206

[TCP] Establishing connection to 142.250.183.206 on port 80
[TCP] SYN
[TCP] SYN-ACK
[TCP] ACK
[TCP] Connection Established

[Reverse Proxy] Checking routing rules...
[Reverse Proxy] Forwarding request to backend: 192.168.1.100:80

[Web Server] Processing request: /
[Web Server] 200 OK

[Browser] Receiving Response
[Browser] Status Code: 200
[Browser] Rendering page...

===== PAGE CONTENT =====
```

- **Learning Outcome:**

1. Understand the complete flow of a URL request from browser to web server.
2. Explain how DNS resolves domain names into IP addresses.
3. Demonstrate the TCP 3-way handshake and HTTPS TLS process.
4. Analyze the role of reverse proxy in request routing and SSL termination.
5. Implement a basic simulation of web communication using Python.